

Integration Contracts

For each external system the platform integrates with: purpose, auth, key API calls, error handling, env vars, and a re-wiring checklist for a new environment.

1. Amazon Connect Campaigns V2

Boto3 client: `boto3.client("connectcampaignsv2")` (wrapped by `vip_shared.infrastructure.persistence.outbound_campaigns_client.OutboundCampaignsClient`).

Purpose: Create, start, stop, describe, and poll outbound dial campaigns.

Authentication: Lambda execution role with `connect-campaigns:*` actions on `arn:aws:connect-campaigns:{region}:{account}:campaign/*`.

Key API calls (executor.py + OutboundCampaignsClient):

Operation	Used in	Notes
CreateCampaign	<code>_create_campaign_only</code> , <code>_start_one_campaign</code>	Includes <code>source.customerProfilesSegment.segmentDefinitionArn</code> , <code>channelSubtypeConfig.telephony</code>
StartCampaign	<code>_start_one_campaign</code> , <code>_prestart_plan</code>	Requires <code>schedule.startTime >= now + 6m</code>
StopCampaign	<code>_expire_bucket</code> , <code>abort_run</code> , 19:00 cutoff handler	Idempotent on already-stopped campaigns
GetCampaignState	<code>tick()</code> polling	Returns enum: <code>INITIALIZED</code> , <code>RUNNING</code> , <code>STOPPED</code> , <code>COMPLETED</code> , <code>FAILED</code>
DescribeCampaign	Diagnostic	Optional, not on hot path
DeleteCampaign	Cleanup (manual)	Not called automatically
TagResource	<code>_start_one_campaign</code>	Tags include <code>planId</code> , <code>runId</code> , <code>environment</code>

Error handling:

- `InvalidStateException` on `StopCampaign` → swallow (campaign already terminal)
- `ResourceNotFoundException` → treat as completed; record `exitReason=stopped`
- `ValidationException` containing "Operation is not valid" → permanent (instance not segment-driven); raise alert
- `ThrottlingException` → retry with exponential backoff (boto3 default)

Env vars: `CONNECT_INSTANCE_ID`

Re-wiring checklist (new env): 1. Provision Connect instance with segment-driven dialing. 2. Set `CONNECT_INSTANCE_ID` env var on api-plans + api-campaigns Lambdas. 3. Verify `connect-campaigns:*` IAM permissions on Lambda execution role. 4. Confirm at least one `campaign-{STATE}` contact flow exists (or rely on auto-creation in `resolve_campaign_flow_arn`).

2. Amazon Customer Profiles

Boto3 client: `boto3.client("customer-profiles")` (wrapped by `vip_shared.infrastructure.persistence.customer_profiles_client.CustomerProfilesClient`).

Purpose: Define dynamic segments over patient profiles, feed them to Connect Campaigns V2.

Authentication: IAM `profile:CreateSegmentDefinition`, `profile:GetSegmentDefinition`, `profile>DeleteSegmentDefinition`, `profile:ListSegmentDefinitions`, `profile:TagResource` scoped to `arn:aws:profile:{region}:{account}:domains/{PROFILES_DOMAIN_NAME}/*`.

Key API calls:

Operation	Used in	Notes
<code>CreateSegmentDefinition</code>	<code>_start_one_campaign</code> , <code>_prestart_plan</code>	Name encoded as <code>{state}-{group}-{timestamp}</code>
<code>GetSegmentDefinition</code>	Reconcile loop	Confirms segment exists + member count
<code>DeleteSegmentDefinition</code>	Manual cleanup	Not called automatically; see TD-013
<code>ListSegmentDefinitions</code>	Admin UI listing	Domain-scoped
<code>TagResource</code>	Creation	<code>planId</code> , <code>runId</code> , <code>campaignId</code> tags

Error handling:

- `BadRequestException` → invalid filter; raise to caller, mark `exitReason=creation_failed`
- `ResourceNotFoundException` during reconcile → retry up to `reconcileRetryLimit`
- Empty member count after retries exhausted → `exitReason=skipped_empty`

Env vars: `PROFILES_DOMAIN_NAME`

Re-wiring checklist: 1. Create Customer Profiles domain. 2. Ingest patient profiles with required attribute structure (state location, segment group). 3. Set `PROFILES_DOMAIN_NAME`. 4. Grant Lambda role `profile:*` on the domain.

3. Amazon Connect Contact Flows

Boto3 client: `boto3.client("connect")`.

Purpose: Resolve and (if missing) create canonical CAMPAIGN-typed contact flows. Used only on the slow path during flow ARN resolution.

Authentication: `connect:ListContactFlows`, `connect:DescribeContactFlow`, `connect:CreateContactFlow`, `connect:TagResource` ON `arn:aws:connect:{region}:{account}:instance/{CONNECT_INSTANCE_ID}/*`.

Key API calls:

Operation	Used in	Notes
<code>ListContactFlows</code>	<code>resolve_campaign_flow_arn</code> , <code>resolve_journey_flow_arn</code>	Filtered by flow type CAMPAIGN
<code>CreateContactFlow</code>	Auto-create canonical flow	Body is a minimal published flow document
<code>DescribeContactFlow</code>	Diagnostic	Optional

Operation	Used in	Notes
TagResource	Creation	environment tag

Naming convention:

- `campaign-{STATE}` for MANAGED delivery (e.g. `campaign-NY`)
- `Test-Journey-Flow` for JOURNEY delivery (single flow for all states)

Re-wiring checklist: 1. Either pre-create one CAMPAIGN flow per state with the canonical name, OR allow auto-creation (needs `connect:CreateContactFlow`). 2. Pre-create `Test-Journey-Flow` for journey deliveries.

4. ElastiCache Redis

Client: `redis-py` via `vip_shared.infrastructure.persistence.redis_lead_source.RedisLeadSource`.

Purpose: Source of truth for lead phone numbers and their segment group labels. Populated by an out-of-scope feeder pipeline.

Authentication: VPC-only access. Lambda is attached to the Redis VPC; security group allows ingress from the Lambda SG.

Key operations:

Operation	Notes
HSCAN <code>wait_list:{team}:list</code>	Batched scan with optional MATCH pattern
HGETALL <code>wait_list:{team}:list</code>	Full read (avoid for large lists; prefer HSCAN)
Server-side filtering	<code>RedisLeadSource.fetch_for_filters(state_codes, groups)</code> deserialises each lead value and filters in Python

Error handling:

- `ConnectionError` → Lambda may have lost VPC attachment; retry once then abort
- Timeout → 5-minute Lambda timeout is the upper bound; large lists should chunk

Env vars: `REDIS_HOST`, `REDIS_PORT`, `TEAM`

Re-wiring checklist: 1. Provision Redis in same VPC/AZ as Lambdas. 2. Open security group ingress from Lambda SG. 3. Set `REDIS_HOST`, `REDIS_PORT`, `TEAM`. 4. Validate `wait_list:{team}:list` is populated by feeder.

5. EventBridge Scheduler

Boto3 client: `boto3.client("scheduler")` and `boto3.client("events")` (see `scheduler_manager.py` — uses `events:PutRule/PutTargets`).

Purpose: Drive the tick loop. One rule per active bucket plus a `prestart_check` rule.

Authentication: `events:PutRule`, `events:PutTargets`, `events:RemoveTargets`, `events>DeleteRule` on `arn:aws:events:{region}:{account}:rule/vip-plan-*` and `arn:aws:events:{region}:{account}:rule/vip-sched-*`. PLUS `lambda:AddPermission/RemovePermission` on the `api-plans` function ARN.

Rule naming:

- `vip-plan-{runId}-{bucketIndex}` — per-bucket tick (1/min)
- `vip-sched-prestart-check` — `prestart_check` rule (1/min, account-wide)

Lifecycle:

1. `_activate_warming_bucket` or `_start_bucket` → `PutRule` `rate(1 minute)` + `PutTargets` (api-plans Lambda) + `AddPermission`
2. `tick` runs each minute, polls Connect, advances state
3. `_advance_bucket` or `_expire_bucket` → `RemoveTargets` + `DeleteRule` + `RemovePermission`

Error handling:

- `ResourceNotFoundException` on `DeleteRule` → already cleaned up, swallow
- Orphan rules (post-incident) require manual janitor; on the backlog

Re-wiring checklist: 1. Grant Lambda role `events:Put*/events>Delete*/lambda:AddPermission` on its own ARN. 2. Pre-create `vip-sched-prestart-check` rule pointing to api-plans Lambda OR allow code to create it on first call.

6. Amazon Cognito

Purpose: Authenticate operators into the Admin UI.

Pool: `us-east-1_MeEkw04P4` (`vip-admin-ui-pool`)

Configuration:

- MFA: TOTP enforced (HIPAA requirement)
- Password policy: 12-char min, all complexity classes
- Token lifetimes: access 1h, refresh 30d, id 1h
- Advanced security features: enabled (adaptive auth)
- Idle session timeout: 15 minutes (HIPAA)

Integration: Admin UI uses Cognito Hosted UI or `amplify-auth`. JWT in `Authorization` header on every API call. `vip_shared.application.http.extract_caller(event)` decodes claims (`sub` , `email`) from the JWT (validated upstream by API Gateway).

Re-wiring checklist: 1. Create user pool with same MFA/password/idle config. 2. Configure app client; set callback URLs to CloudFront distribution. 3. Update Admin UI `VITE_COGNITO_USER_POOL_ID` and `VITE_COGNITO_CLIENT_ID`. 4. Configure API Gateway / Lambda authorizer to validate Cognito JWTs.

7. Amazon SNS — Plan Alerts

Topic: `arn:aws:sns:us-east-1:165505826690:vip-plans-alerts` (imported by ARN, not CDK-managed — see ADR-005).

Purpose: Operational alerts for failed runs, stuck runs, and pre-warm exhaustion. Subscribers are operator email addresses.

Authentication: Lambda role grants `sns:Publish` on the topic.

Key calls: `Publish(TopicArn, Subject, Message)` from `executor.py` on:

- Stuck run detected (`> 4 h` running per `_STUCK_RUN_HOURS`)
- Pre-warm exhausted retries
- Run transitioned to `failed`

Re-wiring checklist: 1. Create topic `vip-plans-alerts` manually (CFN cannot do it under the boundary). 2. Subscribe operator emails (`aws sns subscribe`). 3. Set `SNS_ALERTS_TOPIC_ARN` env var. 4. Verify Lambda role has `sns:Publish` on the topic ARN.

8. Amazon CloudWatch — Logs & Metrics

Log group: `/aws/lambda/vip-admin-ui-api-plans` (1-year retention, CMK encrypted).

Logger: `vip_shared.infrastructure.telemetry.structured_logger.StructuredLogger` emits one-line JSON records. PHI fields (`phone`, `phone_number`, `first_name`, `last_name`, `fullname`) are auto-replaced with `{first_name}_hash` (SHA-256, first 12 chars). `lead_id` becomes `lead_id_hash` + `lead_id_prefix`.

Common log queries (CloudWatch Logs Insights):

```
# Errors in last hour
fields @timestamp, event, errorDetail, planId, runId
| filter level = "ERROR"
| sort @timestamp desc
| limit 100

# Tick latency p95
fields @timestamp, event, durationMs
| filter event = "tick_complete"
| stats pct(durationMs, 95) by bin(5m)
```

Custom metrics (PutMetricData):

Metric	Namespace	Dimensions	Purpose
StuckRunDetected	VipConnect/Plans	planId	Alarmable; threshold 1
PrewarmFailure	VipConnect/Plans	planId	Alarmable
CampaignSkippedEmpty	VipConnect/Plans	state	Informational

Re-wiring checklist: 1. CDK creates encrypted log groups; ensure CMK is in same region. 2. Add CloudWatch Alarms on `StuckRunDetected >= 1` and `PrewarmFailure >= 1` → SNS `vip-plans-alerts`.

Credentials Inventory

Credential	Storage	Rotation	Notes
AWS access keys (developers)	AWS SSO / <code>~/.aws/config</code>	90d	<code>--profile production</code>
Lambda execution roles	IAM, scoped per Lambda	n/a	Permissions-bounded by <code>EngineeringPermissionBoundary</code>
Connect instance	Console-provisioned	n/a	

Credential	Storage	Rotation	Notes
			Instance ID is environment config, not a secret
Cognito app client secret	None (public SPA)	n/a	SPA uses PKCE; no client secret
CMK (DATA_KEY_ARN)	KMS	Annual rotation	CDK-managed
Redis password	None today	n/a	VPC-only access, security group restricted; consider AUTH token rotation
GitHub repo	Org account	n/a	PAT auth, branch protection

Critical Migration Notes

When porting this platform to a new AWS account / region:

1. **Connect Instance is region-locked.** The instance UUID is region-specific. A new region requires a new Connect instance.
2. **Customer Profiles domains are region-locked.** Create one per region.
3. **Lambda Layer version pinning.** Re-publish `vip_shared` layer and update every `api-*` function to point at the new ARN. Layers do NOT replicate across regions.
4. **SNS topic must be hand-created** before `cdk deploy` (the boundary blocks CFN topic creation). Cf. ADR-005.
5. **EventBridge rule names** are constant (`vip-plan-...` , `vip-sched-...`). If two environments share an account, rule names will collide.
6. **CloudFront + S3 bucket name is global.** Pick a unique bucket name in new accounts.
7. **HIPAA: every BAA-eligible service must be re-listed** in the new account's BAA before processing PHI.